

THERMAGUARD AI

AI-Powered Satellite Intelligence for Industrial Fire Detection

Smart India Hackathon · Problem Statement 26162

NTRO · Disaster Management Theme · v1.0

NASA FIRMS Integration

5-Class ML Classifier

Interactive GIS Dashboard

What is THERMAGUARD AI?

Satellite systems like NASA FIRMS can detect thermal anomalies, but they cannot tell you **what** is causing them. THERMAGUARD AI solves this by combining:

- NASA FIRMS satellite thermal data
- OpenStreetMap industrial infrastructure context
- Random Forest Machine Learning classifier

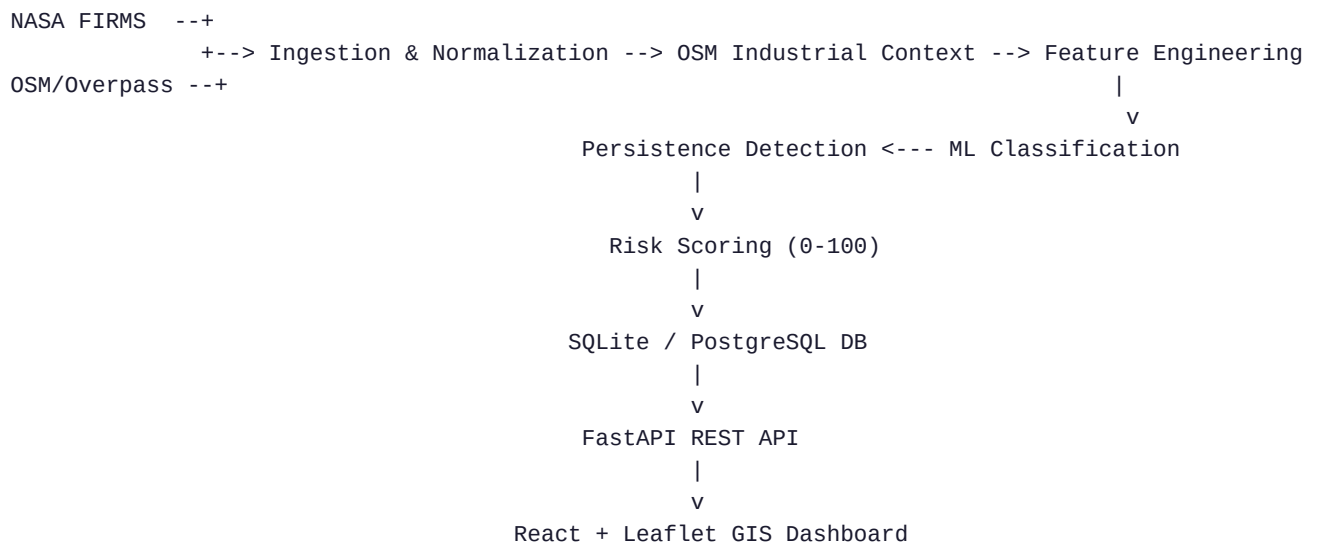
...to classify every thermal event, detect persistent sources, calculate a transparent risk score, and display everything on an interactive GIS dashboard.

The system runs entirely on bundled demo data if NASA FIRMS / Overpass are unavailable — the dashboard is never blank.

Fire Categories Detected

#	Category
1	Industrial Fire
2	Natural / Forest Fire
3	Agricultural Fire
4	Gas Flare / Persistent Thermal Source
5	Other / Unknown

How It Works



For every thermal detection, the system:

1. Normalizes raw detection data (handles missing fields gracefully)
2. Finds nearest industrial facility + counts nearby facilities (OSM data)
3. Builds feature vector and classifies with trained Random Forest
4. Checks for repeated detections in same area (persistence detection)

- 5. Computes transparent 0–100 risk score with plain-language reasons
- 6. Stores and serves through FastAPI → React/Leaflet dashboard

Technology Stack

Layer	Technology
Frontend	React 18, Vite, React-Leaflet, Recharts, Axios
Backend	Python, FastAPI, Pydantic, SQLAlchemy, Uvicorn
ML	scikit-learn (Random Forest), Pandas, NumPy
Database	SQLite (default, zero setup) · PostgreSQL/PostGIS-ready
GIS	Leaflet, OpenStreetMap tiles

Features

- FIRMS-style thermal ingestion with automatic demo-data fallback
- OSM/Overpass industrial infrastructure lookup with demo-data fallback
- 5-class ML fire classification with confidence + full probability distribution
- Rule-based heuristic fallback classifier if trained model is missing
- Persistence detection (repeated detections within configurable radius)
- Transparent, explainable risk scoring (0–100) with itemised reasons
- Interactive Leaflet map: colour-coded event markers, industrial sites, click-to-inspect
- Command-centre dashboard: stat cards + 4 analytics charts
- High / Critical alert panel with map-focus on click
- Filtering by classification, risk level, persistence, industrial proximity
- Graceful loading / empty / error states throughout
- DEMO MODE indicator whenever live external data is not in use

Project Structure

```
thermaguard-ai/  
  backend/      FastAPI app, services (data/ai/risk), DB models  
  ml/           Training script + prediction sanity-check CLI  
  frontend/     React + Vite dashboard  
  data/         Demo FIRMS/industrial-site CSVs + generator script  
  models/       Trained model (fire_classifier.pkl – gitignored)  
  tests/        Backend unit tests  
  requirements.txt  
  .env.example  
  README.md
```

Quick Start — New Laptop / First Time

Prerequisites

- Python 3.11+ → <https://python.org/downloads>
- Node.js 18+ → <https://nodejs.org>

1. Clone / Copy Project

```
git clone <your-repo-url>
# OR copy the entire Thermaguard-AI folder
```

2. Install Backend Dependencies

```
python -m venv .env
source .env/bin/activate      # Windows: .env\Scripts\activate
pip install -r requirements.txt
cp .env.example .env          # safe defaults — works out of the box
```

3. Train / Verify ML Model

```
ls models/                    # should show fire_classifier.pkl
# If missing:
python ml/train.py
```

4. Start Backend

```
cd backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
# Wait for: INFO: Application startup complete.
# Backend: http://localhost:8000
# API Docs: http://localhost:8000/docs
```

5. Install and Start Frontend

Open a **new terminal** (keep backend running):

```
cd frontend
npm install
npm run dev
# Wait for: Local: http://localhost:5173/
# Open: http://localhost:5173
```

Every Time — Run Both Servers

```
Terminal 1 (Backend):  cd backend  → uvicorn main:app --port 8000
Terminal 2 (Frontend): cd frontend → npm run dev
```

Environment Variables

See **.env.example** — every value has a safe default. The app runs **fully offline** with none of them set.

Variable	Required	Purpose
NASA_FIRMS_API_KEY	Optional	Live satellite data (demo CSV used if missing)
OVERPASS_URL	Optional	Live OSM queries (demo CSV used if missing)
DATABASE_URL	Optional	PostgreSQL connection (defaults to local SQLite)
CORS_ORIGINS	Optional	Allowed frontend origins

Database: No manual setup needed. SQLite (thermaguard.db) is created automatically on first run.

ML Training

The bundled **models/fire_classifier.pkl** is already trained. To retrain:

```
cd ml
python train.py      # retrains → overwrites models/fire_classifier.pkl
python predict.py    # sanity-checks example events
# Prints: accuracy, confusion matrix, feature importances
```

Adding More Training Data

Edit **data/fires.csv** and add rows:

```
id,latitude,longitude,brightness,frp,confidence,acquisition_time,source,true_class_demo_only
TG-9001,22.35,69.82,321.1,47.6,90.6,2026-09-10T10:00:00Z,MODIS,Industrial Fire
```

Valid labels: Industrial Fire | Natural / Forest Fire | Agricultural Fire | Gas Flare / Persistent Thermal Source | Other / Unknown

After adding data: **python ml/train.py**, then restart backend.

API Reference

Base URL: <http://localhost:8000> | Interactive docs: <http://localhost:8000/docs>

Method	Endpoint	Description
GET	/health	Service + demo-mode status
GET	/api/events	List events (filterable)
GET	/api/events/stats	Dashboard summary statistics
GET	/api/events/alerts	High / Critical events only
GET	/api/events/{id}	Single event detail
GET	/api/events/{id}/risk	Risk score + reasons
GET	/api/industrial-sites	All industrial infrastructure
POST	/api/events/analyze	Run pipeline on ad-hoc coordinate
POST	/api/data/refresh	Re-run ingestion + classification

Risk Scoring Formula

Factor	Points
Industrial Fire classification	+30
FRP > 300 MW	+20
FRP 100–300 MW	+10

Factor	Points
Distance to industry < 1 km	+25
Distance 1–5 km	+15
Persistent source	+15
Confidence > 80%	+10

Level	Score Range
Critical	>= 80
High	>= 60
Medium	>= 35
Low	< 35

Demo Mode

When NASA_FIRMS_API_KEY is unset or FIRMS/Overpass calls fail, the backend automatically falls back to data/fires.csv and data/industrial_sites.csv.

- DEMO MODE badge — using fallback CSV data
- LIVE DATA badge — both external sources successfully reached

No configuration needed to demo the full flow offline.

Troubleshooting

Problem	Fix
pip not found	Use pip3
python not found	Use python3
Port 8000 busy	Mac: kill \$(lsof -t -i:8000)
ModuleNotFoundError	pip install -r requirements.txt
Map not loading	Check backend is running on port 8000
Model not found	python ml/train.py
npm not found	Install Node.js from nodejs.org

Dataset Transparency (Important for Judges)

Data	Status
data/fires.csv	SYNTHETIC — shaped like real NASA FIRMS output (same fields, realistic ranges). Marked true_

Data	Status
data/industrial_sites.csv	REAL — curated named Indian industrial facilities (refineries, steel plants, power stations, mines, L
ML training data	SYNTHETIC — generated from the feature schema with documented, physically-motivated distrib
Live integration	WIRED & WORKING — NASA_FIRMS_API_KEY and Overpass queries activate automatically wh

Limitations

- ML model trained on synthetic data — treat outputs as proof-of-concept, not certified detection
- Risk score is an explainable heuristic, not a scientifically validated model
- Persistence detection uses simple radius + count (no satellite revisit-time accounting)
- No authentication layer — out of scope for this prototype

Future Improvements

- Train on real labelled FIRMS + verified industrial incident data
- PostGIS-backed spatial queries instead of in-memory haversine loops
- Sentinel-2 satellite imagery overlay for visual event inspection
- Historical trend analytics and facility-level risk profiles
- Role-based access and audit logging for operational deployment
- Push / SMS alerting for Critical events

THERMAGUARD AI is a hackathon prototype. Not an officially deployed NTRO system and makes no claim of certified accuracy.

SIH 2024 · PS-26162 · Disaster Management · v1.0